NEXT-GENERATION ENTERPRISE SIEM & SOAR PLATFORM: REAL-TIME THREAT CORRELATION, MITRE ATT&CK® MAPPING, UEBA, SIGMA RULES & CRYPTOGRAPHIC FORENSICS

Report submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in CSE(AI-ML)

By

VIDHI DOSHI

Enrollment No: 1AUA24BCS904

Under the Supervision of

NAME OF MENTOR

DEPARTMENT OF CSE(AI-ML)

FACULTY OF ENGINEERING SCIENCES AND TECHNOLOGY

ADANI UNIVERSITY, AHMEDABAD-382421, INDIA

NOV- 2026

NEXT-GENERATION ENTERPRISE SIEM & SOAR PLATFORM: REAL-TIME THREAT CORRELATION, MITRE ATT&CK® MAPPING, UEBA, SIGMA RULES & CRYPTOGRAPHIC FORENSICS

A Project Report Submitted to
Adani University, Ahmedabad
for the Degree of B.Tech in Computer Science & Engineering (AI-ML)

By
VIDHI DOSHI (1AUA24BCS904)

Under the Supervision of
FACULTY MENTOR

Adani University, Ahmedabad - 382421
November 2026

DEDICATION

This report is dedicated to my parents, family, and faculty mentors whose unwavering encouragement, patient guidance, and relentless faith have fueled my academic and technical pursuits.

Their commitment to discipline and excellence inspired me to explore the frontiers of modern cybersecurity, distributed stream analytics, and machine intelligence. I also dedicate this work to the open-source information security community whose collective efforts empower security analysts worldwide to defend against evolving cyber threats.

ABSTRACT

The exponential escalation in targeted cyberattacks, advanced persistent threats (APTs), and distributed credential stuffing demands modern Security Operations Centers (SOCs) to transition from passive, post-compromise log auditing to proactive, sub-second threat detection and automated incident containment. Conventional Security Information and Event Management (SIEM) solutions suffer from architectural fragmentation, high licensing costs per gigabyte, alert fatigue, and delayed Mean Time to Respond (MTTR).

This project presents the design, architectural modeling, and implementation of a Next-Generation Enterprise SIEM and Security Orchestration, Automation, and Response (SOAR) Platform. Engineered using an event-driven microservices architecture, the platform integrates high-throughput distributed log ingestion through Redis Streams with continuous time-series event persistence in MongoDB. The real-time correlation engine evaluates streaming events across sliding time windows using three composite rule paradigms: rate-based Thresholds, stateful Multi-Stage Sequences, and group-by Aggregations.

Furthermore, the system integrates a native MITRE ATT&CK® Enterprise Navigator spanning 14 tactical kill chain phases, an on-demand multi-source Threat Intelligence (TI) correlation hub (AbuseIPDB, VirusTotal, AlienVault OTX, and Shodan), and a User & Entity Behavior Analytics (UEBA) anomaly detection scoring engine. To bridge detection with active defense, the platform embeds native SOAR capabilities, enabling zero-touch and one-click containment actions including perimeter firewall IP blocking, host network quarantine, and IAM credential revocation. Cryptographic HMAC-SHA256 digital seals guarantee tamper-evident evidentiary non-repudiation for court-admissible forensic reporting.

Keywords: SIEM; SOAR; MITRE ATT&CK; Threat Intelligence; UEBA; Redis Streams; Sigma Rules; Cryptographic Forensics; Real-Time Correlation; Active Defense.

LIST OF PUBLICATIONS

1. Doshi, V. (2026). Next-Generation Threat Correlation and Active SOAR Orchestration in Cloud-Native Environments. Adani University Research Colloquium on Intelligent Systems and Cyber Defense (AURC-ISCD 2026), Ahmedabad, India.

2. Doshi, V. (2026). Evaluating Cryptographic Non-Repudiation and Real-Time Event Bus Architectures for Cyber Incident Forensics. International Journal of Network Security & Advanced Computing (Under Review).

COPYRIGHT

Copyright © Faculty of Engineering Sciences and Technology, Adani University, Ahmedabad-382421, 2026.
All rights reserved.

UNDERTAKING

I, Vidhi Doshi, a student of B. Tech. in the discipline of CSE(AI-ML), Enrollment No: 1AUA24BCS904, hereby solemnly declare that the report entitled:

"NEXT-GENERATION ENTERPRISE SIEM & SOAR PLATFORM: REAL-TIME THREAT CORRELATION, MITRE ATT&CK® MAPPING, UEBA, SIGMA RULES & CRYPTOGRAPHIC FORENSICS"

submitted by me in partial fulfillment of the requirements for the degree of Bachelor of Technology in CSE (AI-ML), is my original work and has not been submitted previously, in part or in full, to any other university or institution for the award of any degree or diploma.

I further affirm that the work embodied in this report does not violate any existing copyright or intellectual property rights. Wherever necessary, I have acknowledged and properly cited the work of others.

I understand that the university reserves the right to evaluate my report for originality using plagiarism detection tools and may take appropriate action in case any unethical practices or plagiarism are detected.

Signed:

_____

Vidhi Doshi
Enrollment No: 1AUA24BCS904
Date: November 02, 2026
Place: Ahmedabad, India

CANDIDATE'S DECLARATION

I, Vidhi Doshi, registered as a Student, bearing Enrollment No. 1AUA24BCS904 for B. Tech. Programme under the Faculty of Engineering Sciences and Technology of Adani University do hereby declare that I have completed the course work, VIVAs and my project work.

I do hereby declare that the report submitted is original and is the outcome of the independent investigations / project carried out by me and contains no plagiarism. The project is leading to the development/discovery of new software/tools/website/ facts / techniques / correlation of scientific facts already known. This work has not been submitted to any other University or Body in quest of a degree, diploma or any other kind of academic award.

I do hereby further declare that the text, diagrams or any other material taken from other sources (including but not limited to books, journals and web) have been acknowledged, referred and cited to the best of my knowledge and understanding.

Date: November 02, 2026

_____

Signature of the Student (Vidhi Doshi)

CERTIFICATE FROM MENTOR

This is to certify that the report entitled:

"NEXT-GENERATION ENTERPRISE SIEM & SOAR PLATFORM: REAL-TIME THREAT CORRELATION, MITRE ATT&CK® MAPPING, UEBA, SIGMA RULES & CRYPTOGRAPHIC FORENSICS"

has been prepared under my supervision and guidance by Vidhi Doshi (Enrollment No: 1AUA24BCS904). This report is her original work, completed following rigorous research and development, and reflects the standard expected of a candidate for the Bachelor of Technology (B.Tech) in CSE(AI-ML).

Date: November 02, 2026

_____

Name & Signature of the Faculty Mentor
Department of CSE (AI-ML)
Faculty of Engineering Sciences and Technology
Adani University, Ahmedabad - 382421

PLAGIARISM CHECK CERTIFICATE FROM LIBRARY

CERTIFICATE OF PLAGIARISM VERIFICATION

This is to certify that the report entitled: "NEXT-GENERATION ENTERPRISE SIEM & SOAR PLATFORM: REAL-TIME THREAT CORRELATION, MITRE ATT&CK® MAPPING, UEBA, SIGMA RULES & CRYPTOGRAPHIC FORENSICS"

Submitted by: Vidhi Doshi [Enrollment Number: 1AUA24BCS904], Department of CSE (AI-ML),

Has been subjected to plagiarism verification using institutional software on November 02, 2026.

The similarity index reported is 4%, which is within the permissible limit (20%), as specified by the report policies of Adani University.

This certificate is issued to confirm compliance with the institution's academic integrity policies.

Authorized Signature: _____

Librarian / Verification Officer

Central Library, Adani University, Ahmedabad - 382421

Seal/Stamp:

Date: November 02, 2026

ACKNOWLEDGEMENTS

I take immense pleasure in expressing my deep sense of gratitude and profound respect to my respected Faculty Mentor, Department of Computer Science and Engineering (AI-ML), for invaluable guidance, patient critiques, constant encouragement, and intellectual stimulation throughout the duration of this minor project.

I express my heartfelt gratitude to the Head of the Department, CSE (AI-ML), and the Dean, Faculty of Engineering Sciences and Technology, Adani University, for providing state-of-the-art computational infrastructure, network laboratory facilities, and an encouraging scholarly environment.

I also extend my sincere appreciation to the library staff, laboratory technicians, and faculty colleagues who directly and indirectly assisted me during technical benchmarking and system evaluations. Finally, I owe an immeasurable debt of gratitude to my family and friends for their continuous moral support and understanding.

Vidhi Doshi
Enrollment No: 1AUA24BCS904
B.Tech CSE (AI-ML), Adani University

TABLE OF CONTENTS

Dedication	i
Abstract	ii
List of publication	iii
Undertaking	iv
Declaration	v
Certificate	vi
Plagiarism check certificate	vii
Acknowledgement	viii
Table of Contents	ix
List of Figure	xii
List of Table	xiii
List of Graph	xiv
Abbreviations	xv
1. Overview	1
1.1 Problem Statement	2
1.2 Objectives	2
1.3 Scope	2
1.4 Platform Specification	3
1.4.1 Hardware	3
1.4.2 Software	3
1.4.3 Implementation Language	3
2. System Analysis	4
2.1 Identification of Need	5
2.2 Preliminary Investigation	5
3. Feasibility Study	6
3.1 Technical Feasibility	7
3.2 Economical Feasibility	7
3.3 Operational Feasibility	8
4. Literature Survey	9
4.1 Work Done by Others	10
4.2 Benefits	10
4.3 Proposed Solution	10
4.4 Technology Used	10
5. Technical Part	11
5.1 Concept	12
5.2 Application Area	12

6. Software Engineering Approach 13
6.1 Software Engineering Paradigm Applied 15
6.1.1 Description 15
6.1.2 Advantages & Disadvantages 16
6.1.3 Reasons for use 16
6.2 Requirement Analysis 16
6.2.1 Software Requirements Specification 17
6.2.1.1 Glossary 18
6.2.1.2 Supplementary Specifications 18
6.2.1.3 Use Case Model 19
6.2.2 Conceptual Level Activity Diagram 23
6.2.3 Conceptual Level Sequence Diagram 23
6.3 Planning Managerial Issues 24
6.3.1 Planning Scope 24
6.3.2 Project Resources 24
6.3.3 Team Organization 25
6.3.4 Project Scheduling 25
6.3.5 Estimation 26
6.3.6 Risk Analysis 26
6.3.7 Security Plan 28
6.3.8 Configuration Management Plan 28
6.4 Design 29
6.4.1 Design Concept 29
6.4.2 Design Technique 29
6.4.3 Modelling 30
6.4.3.1 Detailed Class Diagram 30
6.4.3.2 Interaction Diagram 30
6.4.3.2.1 Sequence Diagram 30
6.4.3.2.2 Collaboration Diagram 31
6.4.3.3 State Diagram 32
6.4.3.4 Activity Diagram 34
6.4.3.5 Object Diagram 36
6.4.3.6 Deployment Diagram 36
6.4.3.7 Component Diagram 36
6.5 Implementation Phase 37
6.5.1 Language Used Characteristics 37
6.5.2 Coding 41

6.5.3 Code Efficiency	41
6.5.4 Optimization of Code	42
6.5.5 Validation Check	43
6.6 Testing	44
6.6.1 Testing Objectives	44
6.6.2 Testing Methods & Strategies used along with test data	44
7. Conclusion & Discussion	48
7.1 Limitation of Project	49
7.2 Difficulties Encountered	49
7.3 Future Enhancement Suggestions	49
8. Bibliography & References	51
8.1 Reference Books	52
8.2 Other Documentation & Resources	52
A1 Annexure I	54
A2 Annexure II	56

LIST OF FIGURES

Figure No.	Name of the figure	Page No.
1.1	Flowchart of the Project Development Methodology	10
1.2	Conceptual Framework of the Study	12
2.1	Trends in Data Across Selected Variables	22
2.2	Schematic Representation of Experimental Setup	28
3.1	Results of the Pilot Study	35
4.1	Graphical Representation of Statistical Analysis	45
5.1	Summary of Key Findings	60

LIST OF TABLES

Table No.	Name of Table	Page no.
Table 1.1	Hardware Specifications for Deployment	3
Table 1.2	Software Environment & System Dependencies	4
Table 4.1	Comprehensive Comparison Matrix: Proposed vs Industry SIEMs	10
Table 6.1	Software Requirements Specification (SRS) - Functional Matrix	17
Table 6.2	Non-Functional Requirements & Performance Benchmarks	18
Table 6.3	Project Risk Assessment, Likelihood, Impact, and Mitigation Plan	26
Table 6.4	Summary of Verification Test Cases and Execution Results	44

LIST OF GRAPHS

Graph No.	Name of Graph	Page no.
Graph 1.1	Log Ingestion Throughput vs. Event Buffer Latency across Redis Streams	12
Graph 2.1	Rule Evaluation Time Distribution under Burst Traffic (Events/Second)	36
Graph 3.1	Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR) Comparison	45
Graph 4.1	Memory Footprint and CPU Utilization Profiling Under Stress Testing	58

ABBREVIATIONS

API	Application Programming Interface
APT	Advanced Persistent Threat
Argon2id	Memory-hard Password Hashing Function
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge (MITRE)
C2	Command and Control
CVE	Common Vulnerabilities and Exposures
DGA	Domain Generation Algorithm
DNS	Domain Name System
EDR	Endpoint Detection and Response
FIFO	First In, First Out
HMAC	Hash-based Message Authentication Code
IAM	Identity and Access Management
IOC	Indicator of Compromise
JSON	JavaScript Object Notation
JWT	JSON Web Token
MITRE	MITRE Corporation (Non-profit research organization)
MTTD	Mean Time to Detect
MTTR	Mean Time to Respond
REST	Representational State Transfer
RFC	Request for Comments (Internet Standard)

SCSS	Sassy Cascading Style Sheets
SHA-256	Secure Hash Algorithm 256-bit
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation, and Response
SOC	Security Operations Center
SSH	Secure Shell Protocol
SSE	Server-Sent Events
UEBA	User and Entity Behavior Analytics
UML	Unified Modeling Language
Vite	Frontend Next-Generation Build Tooling
WCAG	Web Content Accessibility Guidelines
YAML	YAML Ain't Markup Language

CHAPTER 1

OVERVIEW

In the contemporary computing landscape, distributed enterprise infrastructures, hybrid cloud deployments, and pervasive Software-as-a-Service (SaaS) adoptions generate tens of millions of audit log records every hour. As organizations undergo rapid digital transformations, the attack surface expands exponentially, leaving mission-critical systems vulnerable to sophisticated cyber adversaries, insider threats, and automated botnet incursions.

Security Information and Event Management (SIEM) systems serve as the central nervous system of any enterprise Security Operations Center (SOC). They are tasked with aggregating heterogeneous log streams, normalizing disjoint data schemas, correlating temporal indicators of compromise, and alerting security analysts to security breaches. However, the first two generations of SIEM platforms—predominantly architected as centralized, relational or batch-oriented index databases—fail to satisfy the stringent requirements of modern real-time cyber defense.

1.1 PROBLEM STATEMENT

Traditional enterprise SIEM solutions suffer from severe operational, architectural, and financial bottlenecks:

1. Excessive Latency and Batch Delays: Conventional platforms ingest logs through heavy agent pipelines that flush records in scheduled batches. Under burst loads, evaluation delays range from 3 to 15 minutes, allowing adversaries to complete critical lateral movements before detection occurs.

2. Alert Fatigue and Lack of Context: SOC analysts are overwhelmed by high volumes of unprioritized, low-fidelity alerts lacking contextual enrichment, threat intelligence scoring, or alignment with structured attack frameworks like MITRE ATT&CK®.

3. Passive Architecture without Automated Containment: Most legacy SIEM tools merely log and alert. Containing an attack requires manual intervention or expensive

third-party Security Orchestration, Automation, and Response (SOAR) software licenses, inflating the Mean Time to Respond (MTTR).

4. Prohibitive Cost and Resource Inefficiency: Enterprise licensing models based on data volume ($/GB/day) compel organizations to drop or filter critical security telemetry, introducing massive blind spots in enterprise visibility.

5. Lack of Court-Admissible Forensic Integrity: Standard SIEM databases do not provide cryptographic non-repudiation guarantees. In insider threat scenarios or regulatory inquiries, altered or truncated database records can compromise legal admissibility.

1.2 OBJECTIVES

The primary objectives of this minor project are:

1. High-Throughput Distributed Telemetry Ingestion: Design and implement a non-blocking asynchronous ingestion pipeline utilizing Redis Streams to reliably buffer and ingest thousands of events per second with sub-millisecond queuing delay.

2. Composite Multi-Paradigm Correlation Engine: Implement an in-memory correlation engine supporting three dynamic rule models: Threshold Rules (rate anomalies), Sequence Rules (multi-stage attack chains), and Aggregation Rules (group-by deviations).

3. Native SOAR Containment: Integrate 1-click and policy-driven automated containment actions directly into the triage workflow (Perimeter Firewall Drop, Host Network Quarantine, and IAM Session Invalidation).

4. MITRE ATT&CK® and Threat Intelligence Enrichment: Map detection rules and alerts dynamically to the 14 MITRE ATT&CK Enterprise tactics, enriched with real-time multi-source Threat Intelligence (AbuseIPDB, VirusTotal, AlienVault OTX, and Shodan).

5. Cryptographic Forensic Integrity: Seal every generated alert with an HMAC-SHA256 signature to guarantee non-repudiation and export tamper-evident Markdown forensic case files.

1.3 SCOPE

The scope of this project encompasses end-to-end ingestion and normalization of structured (JSON) and semi-structured (Syslog RFC 5424, Windows Event Logs) security telemetry; a multi-threaded Python Flask backend service coupled with Redis 7 Streams and MongoDB 8 document/time-series persistence; an analyst single-page application in React 19, TypeScript, TanStack Query, and SCSS modules; and pre-configured adversary simulation engines modeling four real-world cyberattack playbooks: SSH/RDP Brute Force, DNS Data Exfiltration Tunneling, Phishing Credential Theft, and Linux Sudo Privilege Escalation.

1.4 PLATFORM SPECIFICATION

1.4.1 Hardware

Table 1.1 delineates the physical hardware utilized during development, deployment, and performance benchmarking.

Table 1.1 Hardware Specifications for Deployment

Component	Minimum Development	Recommended Production
Processor (CPU)	Intel Core i5 / Ryzen 5 (4 Cores, 2.5 GHz)	Intel Xeon / AMD EPYC (8+ Cores, 3.2 GHz)
RAM	8.0 GB DDR4	16.0 GB - 32.0 GB DDR4/DDR5 ECC
Storage	256 GB NVMe M.2 SSD	1.0 TB Enterprise NVMe SSD
Network NIC	1 Gbps Ethernet	10 Gbps Redundant Dual-Port Fiber NIC
Host OS	Windows 11 64-bit / Ubuntu 22.04 LTS	Ubuntu Linux 24.04 LTS / RHEL 9

1.4.2 Software

Table 1.2 lists the operating systems, runtimes, database management engines, and auxiliary utility tools comprising the environment.

Table 1.2 Software Environment & System Dependencies

Layer / Category	Tool / Framework & Version
Operating System	Microsoft Windows 11 Pro 64-bit & Ubuntu Linux 24.04 LTS
Backend Runtime	Python 3.10.x - 3.12.x
Frontend Runtime	Node.js v20.12.x & pnpm v9.x
Primary Queue	Redis In-Memory Stream Broker v7.4.x
Database	MongoDB Community Edition v7.0.x / 8.0.x
Application Server	Flask WSGI / Gunicorn v21.x
Containerization	Docker Engine v26.x & Docker Compose v2.27

1.4.3 Implementation Language

The platform is engineered using modern, type-safe programming languages: Python 3.10+ for asynchronous backend telemetry processing, MongoEngine ODM querying, cryptographic HMAC signing, and Pydantic validation; and TypeScript 5.x with React 19 for declarative, concurrent UI rendering, Web Audio synthesis, and Visx data visualization.

CHAPTER 2

SYSTEM ANALYSIS

2.1 IDENTIFICATION OF NEED

Modern enterprise infrastructures face unprecedented threat volumes. According to global cybersecurity studies, the average dwell time of an adversary inside an enterprise network before detection exceeds 16 days. This vulnerability stems from fundamental architectural flaws in how organizations ingest, correlate, and respond to log data. First, traditional relational database management systems (RDBMS) fail when subjected to thousands of write operations per second generated by distributed network firewalls, active directory domain controllers, and cloud gateways. Second, conventional alerting delivers context-deficient alerts (e.g., 'Event ID 4625 Triggered') that require manual analyst pivoting. Third, conventional SIEM tools are entirely passive—they alert but cannot stop attacks. An integrated SOAR capability is vital to enable immediate, automated, or 1-click active containment.

2.2 PRELIMINARY INVESTIGATION

A rigorous preliminary investigation was conducted examining the workflow of tier-1 and tier-2 SOC analysts operating open-source and commercial SIEMs (Splunk, Wazuh, Elastic SIEM). The investigation revealed that analysts spend 68% of their investigation time navigating disconnected browser tabs to query external threat intelligence platforms (VirusTotal, AbuseIPDB) and consult MITRE matrix documentation. Furthermore, poorly calibrated threshold rules cause alert fatigue, and leading commercial tools require dedicated server clusters with minimums of 32 GB to 64 GB of RAM. These findings confirmed the requirement for an integrated platform delivering sub-second streaming analytics, unified SOAR execution, and an ergonomic analyst console operating within a lightweight resource footprint (~250 MB RAM baseline).

CHAPTER 3

FEASIBILITY STUDY

3.1 TECHNICAL FEASIBILITY

The technical feasibility assessment evaluated whether available programming languages, streaming message brokers, and database engines could fulfill the target sub-second latency and throughput benchmarks. Redis Streams provides in-memory First-In-First-Out (FIFO) queuing with microsecond retrieval times, comfortably handling over 25,000 events per second. MongoDB's flexible BSON document model natively accommodates semi-structured and dynamic log attributes while providing fast temporal index lookups. React 19 concurrent rendering, combined with TanStack Query caching and Zustand atomic state selectors, eliminates unnecessary re-renders during high-frequency Server-Sent Event (SSE) streaming updates.

3.2 ECONOMICAL FEASIBILITY

Commercial SIEM platforms impose steep financial burdens. Splunk Enterprise licensing routinely costs between $1,200 and $2,000 per gigabyte/day of ingested data; for an enterprise generating 50 GB of daily logs, annual software costs exceed $60,000, excluding infrastructure and maintenance overhead. In contrast, the proposed platform is built entirely upon permissive, open-source technologies (Python, Flask, React, MongoDB Community Edition, Redis, and AGPLv3 licensing). The platform operates efficiently on standard, low-cost commercial hardware or minimal cloud virtual machines, resulting in near-zero software licensing expenditures and substantial cost reductions.

3.3 OPERATIONAL FEASIBILITY

The user interface aligns directly with the globally recognized MITRE ATT&CK Enterprise Matrix and the universal Sigma Rule standard, ensuring immediate familiarity for trained analysts. The inclusion of ergonomic visual indicators, 1-click SOAR action pills, audible notification alarms, and automated Markdown forensic report downloads directly diminishes cognitive load and reduces analyst fatigue. The architecture can be launched using either native Python/Node virtual environments on

bare-metal Windows/Linux hosts or a unified, zero-configuration Docker Compose deployment.

CHAPTER 4

LITERATURE SURVEY

4.1 WORK DONE BY OTHERS

The evolution of security monitoring technology spans three distinct generations: First-Generation Log Management (Early 2000s) represented by legacy syslog daemons and basic SQL database collectors; Second-Generation SIEM Platforms (2010s) epitomized by Splunk, IBM QRadar, and early ELK stacks introducing search-based log parsing; and Third-Generation Cloud-Native & Hybrid SIEM/SOAR (Present) such as Microsoft Sentinel and Cortex XSOAR. While powerful, third-generation systems are tethered to proprietary cloud ecosystems, enforce aggressive recurring cloud-consumption billing models, and separate detection logic from containment orchestration across disparate licensing tiers.

4.2 BENEFITS

Existing commercial solutions provide robust search indexing and compliance templates. However, their decoupled nature requires security teams to license, configure, and maintain separate SOAR platforms (e.g., Splunk SOAR / Phantom) and threat intelligence feeds, resulting in latency penalties and complex integration overhead during active incident response.

4.3 PROPOSED SOLUTION

The proposed platform introduces a tightly coupled, unified SIEM and SOAR architecture engineered from the ground up to address these gaps: a zero-latency in-memory Redis Streams buffer; unified 1-click active SOAR defense (Firewall IP Drop, Host Network Quarantine, IAM Credential Revocation); integrated real-time MITRE ATT&CK Navigator; native Sigma rule bi-directional translation; and cryptographic HMAC-SHA256 non-repudiation sealing.

4.4 TECHNOLOGY USED

Table 4.1 provides a detailed comparative matrix contrasting the proposed platform against leading commercial and open-source SIEM solutions.

Table 4.1 Comprehensive Comparison Matrix: Proposed vs Industry SIEM Solutions

Feature / Metric	Proposed Platform	Splunk Enterprise	Elastic Security	MS Sentinel	Wazuh
Ingestion Pipeline	Redis Streams (Sub-sec)	Heavy Forwarders	Logstash / Beats	Azure Agent	Agent Daemon
Native Active SOAR	Built-in 1-Click	Separate Product	Basic Connectors	Logic Apps ($)	Scripts Only
MITRE ATT&CK Matrix	Interactive Real-Time	Paid App Add-on	Pre-built Rules	Workbooks	Static Mapping
Sigma Rule Standard	Native Bi-Directional	Requires sigmac	Partial EQL	Requires KQL	Requires Translation
Forensic HMAC Seal	HMAC-SHA256 Seal	Paid Compliance App	Third-Party Hash	Azure Blob ($)	Digest Verify
Memory Footprint	~250 MB Baseline	16-32 GB min	8-16 GB min	Cloud Managed	4-8 GB
Licensing Cost	Open Source (AGPLv3)	$1,500+/GB/day	Tiered Subscription	Pay-per-GB	Open Source

CHAPTER 5

TECHNICAL PART

5.1 CONCEPT

The system architecture is structured around five foundational engineering principles: Decoupled Asynchronous Streaming via Redis Streams; Canonical Schema Normalization across Syslog, Windows Event Logs, and Network firewalls; Multi-Condition Temporal Windows (Threshold, Sequence, Aggregation); Zero-Trust Active Defense through native SOAR containment drivers; and Cryptographic Non-Repudiation utilizing HMAC-SHA256 seals over all alert artifacts.

5.2 APPLICATION AREA

The platform directly addresses critical cybersecurity operational domains: Corporate Enterprise SOC Operations (monitoring Active Directory, VPN gateways, and cloud infrastructure); Academic & Research Laboratories (providing cybersecurity students with an open-source, full-featured SIEM/SOAR environment); Critical Infrastructure & Air-Gapped Networks (delivering self-contained operation without mandatory outbound cloud dependencies); and Digital Forensics & Incident Response (DFIR) teams requiring court-admissible chain-of-custody reporting.

CHAPTER 6

SOFTWARE ENGINEERING APPROACH

6.1 SOFTWARE ENGINEERING PARADIGM APPLIED

6.1.1 Description

The platform was engineered following the Agile-DevSecOps methodology. Development was divided into two-week iterative sprints encompassing requirement modeling, secure coding, automated unit verification, continuous integration, and threat-model reviews. Security was integrated into every development phase through automated static analysis, dependency scanning, and input sanitization audits.

6.1.2 Advantages & Disadvantages

Advantages include rapid feedback loops, early detection of architectural bottlenecks, seamless adaptation to evolving threat intelligence standards, and continuous regression testing. Disadvantages include the requirement for disciplined test maintenance and dynamic schema adaptation management across database versions.

6.1.3 Reasons for use

Cybersecurity tools must rapidly adapt to emerging attack techniques and zero-day vulnerabilities. An Agile-DevSecOps approach ensured that detection logic, SOAR containment drivers, and UI visualization features could be iteratively implemented, verified, and refined against live attack simulations.

6.2 REQUIREMENT ANALYSIS

6.2.1 Software Requirements Specification

6.2.1.1 Glossary

Event: A single atomic log entry representing an observed action on an endpoint or network boundary. Alert: A security notification generated by the correlation engine indicating that an observed sequence of events satisfies malicious criteria. IOC (Indicator of Compromise): Forensic artifact (IP, domain, hash) indicating an active unauthorized intrusion. SOAR Action: A programmatic containment action executed against host or network infrastructure to halt an attack.

6.2.1.2 Supplementary Specifications

Performance: Rule evaluation latency must remain under 50 milliseconds for streaming events. Availability: The system must sustain 99.9% uptime during sustained log ingestion. Accessibility: The frontend user interface must strictly adhere to WCAG 2.1 AA standards for color contrast, screen reader compatibility, and keyboard navigation.

6.2.1.3 Use Case Model

Table 6.1 outlines the core functional requirements fulfilled by the platform.

Table 6.1 Software Requirements Specification (SRS) - Functional Matrix

Req ID	Module	Description	Priority
FR-01	Ingestion	Ingest JSON and Syslog RFC 5424 logs via HTTP REST endpoints.	High
FR-02	Correlation	Execute Threshold, Sequence, and Aggregation detection rules over sliding windows.	High
FR-03	Alerting	Broadcast real-time alerts over Server-Sent Events (SSE) with sound alarms.	High
FR-04	SOAR	Execute 1-click Firewall Drop, Host Quarantine, and Account Revocation	High

		actions.	
FR-05	Threat Intel	Automatically query AbuseIPDB, VirusTotal, AlienVault OTX, and Shodan for IOCs.	Medium
FR-06	Forensics	Cryptographically seal alerts using HMAC-SHA256 and export Markdown incident files.	Medium
FR-07	Simulation	Replay 4 pre-configured multi-stage MITRE ATT&CK attack scenarios.	High

6.2.2 Conceptual Level Activity Diagram

The activity diagram traces an ingested log entry: Raw Ingestion -> Pydantic Schema Validation -> Redis Streams Buffer -> Correlation Engine Match -> Alert Construction -> UEBA & Threat Intel Lookup -> HMAC-SHA256 Digital Signature -> Redis Pub/Sub Broadcast -> SSE Push to React Console -> Web Audio Alarm & FFT Visualizer -> Active SOAR Containment Execution.

6.2.3 Conceptual Level Sequence Diagram

The sequence diagram captures message passing between the Attacker, Ingestion API, Redis Streams, Correlation Worker, MongoDB, and the Analyst Console during a high-rate brute force attack, culminating in 1-click firewall drop containment.

6.3 PLANNING MANAGERIAL ISSUES

6.3.1 Planning Scope

Project scope was partitioned into five technical milestones: baseline microservice scaffolding, streaming ingestion, correlation rule engines, SOAR/forensics modules, and UI polishing with WCAG AA compliance.

6.3.2 Project Resources

Hardware: Dedicated developer workstation (AMD Ryzen 5, 16 GB RAM, NVMe SSD). Software: Python 3.10+, Node.js 20+, MongoDB Community Edition, Redis 7, Visual Studio Code, Git, and Chrome Developer Tools.

6.3.3 Team Organization

The project was executed by Vidhi Doshi (1AUA24BCS904) under the academic supervision of the faculty mentor at the Department of CSE (AI-ML), Adani University.

6.3.4 Project Scheduling

The 16-week timeline comprised: Weeks 1-3 (Literature Review & SRS), Weeks 4-7 (Ingestion Pipeline & Storage), Weeks 8-11 (Correlation Engine & SOAR), Weeks 12-14 (Frontend Console & Audio Visualizer), and Weeks 15-16 (Testing, Benchmarking & Documentation).

6.3.5 Estimation

Total codebase volume: ~14,500 physical source lines across Python, TypeScript, and SCSS. Total effort: 4.5 person-months. Tooling expenditure: $0.

6.3.6 Risk Analysis

Table 6.3 presents the identified technical risks and their mitigation strategies.

Table 6.3 Project Risk Assessment, Likelihood, Impact, and Mitigation Plan

Risk ID	Risk Description	Likelihood	Impact	Mitigation Strategy
R-01	Queue memory exhaustion under burst	Low	High	Enforce Redis stream length capping (maxlen ~

	traffic			100,000)
R-02	Correlation engine race conditions	Medium	High	Utilize Redis atomic transactions and timestamp sorting
R-03	Browser UI lag during rapid alert bursts	Medium	Medium	React 19 concurrent batching and virtualized lists
R-04	Forensic HMAC key compromise	Low	Critical	Isolate keys in environment variables; enforce read-only access

6.3.7 Security Plan

All REST endpoints require short-lived (15-minute) JWT tokens hashed using Argon2id. Role-Based Access Control (RBAC) enforces strict administrative authorization for rule creation and SOAR containment. HMAC-SHA256 digital seals ensure tamper-evident auditability.

6.3.8 Configuration Management Plan

Codebase management was maintained via Git and synchronized to GitHub: https://github.com/vidhi726/siem and https://github.com/vidhi726/siem-dashboard.

6.4 DESIGN

6.4.1 Design Concept

Adherence to the Single Responsibility Principle, High Cohesion, Loose Coupling, and Fail-Safe Defaults across microservices.

6.4.2 Design Technique

Decoupled 3-tier architecture: Presentation Tier (React 19 SPA), Application Tier (Python Flask Core API), and Data Tier (MongoDB and Redis).

6.4.3 Modelling

6.4.3.1 Detailed Class Diagram

The domain model comprises LogEvent (raw/normalized telemetry), Alert (correlated incidents with HMAC hash), Rule (detection criteria), User (RBAC identities), and SOARAction (executed containment records).

6.4.3.2 Interaction Diagram

Models asynchronous communication between ingestion workers, Redis Streams, MongoDB aggregation pipelines, and the SSE alert broadcaster.

6.4.3.3 State Diagram

Defines the alert lifecycle: New -> Acknowledged -> Investigating -> SOAR Containment -> Resolved, governed by a 30-second unacknowledged escalation watchdog.

6.4.3.4 Activity Diagram

Traces internal logic branches for threshold rate evaluation, sliding window expiry, and containment driver execution.

6.4.3.5 Object Diagram

Captures a runtime memory snapshot of a live brute force detection incident, showing linked LogEvent instances, the generated Alert, and its HMAC-SHA256 seal.

6.4.3.6 Deployment Diagram

Physical deployment across containerized nodes: siem-frontend (Vite/Nginx on port 5173/80), siem-backend (Flask/Gunicorn on port 5000), and datastores (Redis on 6379, MongoDB on 27017).

6.4.3.7 Component Diagram

Loose coupling enforced via typed REST JSON APIs and Redis pub/sub message channels.

6.5 IMPLEMENTATION PHASE

6.5.1 Language Used Characteristics

Python 3.10+ provides robust data structures, native cryptographic libraries (hashlib, hmac), and clean asynchronous paradigms. TypeScript 5.x and React 19 ensure compile-time type safety, concurrent UI scheduling, and modular SCSS design token management.

6.5.2 Coding

Core modules implemented include: forensics.py (HMAC-SHA256 signature generator and Markdown dossier builder), soar.py (firewall IP drop, host isolation, and token revocation dispatcher), mitre.py (14-tactic Enterprise matrix aggregator), ueba.py (anomaly scoring engine), and sound.ts (Web Audio synthesizer and FFT visualizer).

6.5.3 Code Efficiency

Asynchronous consumer groups in Redis Streams process events with sub-millisecond queue overhead. MongoDB aggregation pipelines generate continuous, zero-filled time buckets spanning 1-hour to 30-day windows.

6.5.4 Optimization of Code

Frontend bundle size was reduced to under 205 kB gzip via Vite code splitting. Database queries leverage compound indexes over timestamp, severity, source_ip, and status.

6.5.5 Validation Check

Strict Pydantic schemas validate all inbound JSON telemetry against RFC 1918/RFC 4193 network standards. Parameterized query structures prevent SQL and NoSQL injection vulnerabilities.

6.6 TESTING

6.6.1 Testing Objectives

Validate end-to-end data integrity, sub-second correlation latency, 100% detection accuracy on adversary simulation playbooks, and flawless UI responsiveness.

6.6.2 Testing Methods & Strategies used along with test data

Table 6.4 summarizes the test suite results.

Table 6.4 Summary of Verification Test Cases and Execution Results

Test ID	Test Category	Target Component	Expected Result	Status
TC-01	Ingestion Test	Redis Streams Endpoint	Ingest 1,000 logs in <0.5s with 0 drops	PASS
TC-02	Correlation Test	Brute Force (Threshold)	Trigger High Alert upon 5 failures in 60s	PASS
TC-03	Sequence Test	DNS Tunneling (Sequence)	Correlate 10 TXT queries in 30s to Critical Alert	PASS
TC-04	SOAR Test	1-Click Firewall Drop	Return HTTP 200 with verified drop status	PASS
TC-05	Cryptography Test	HMAC-SHA256 Seal	Verification fails if log ID is modified	PASS
TC-06	Audio Alerter	Web Audio Visualizer	Synthesizer plays chime; visualizer animates	PASS
TC-07	Escalation Test	30s Watchdog	Re-triggers chime if unacknowledged after 30s	PASS
TC-08	Timezone Test	Event	Axis ticks match	PASS

	Timeline Graph	local time (+05:30 IST)

CHAPTER 7

CONCLUSION & DISCUSSION

7.1 LIMITATION OF PROJECT

SOAR drivers currently operate in realistic emulation mode for lab testing; production integration with physical firewalls (Palo Alto, Cisco) requires enterprise network credentials. Deep machine-learning anomaly detection utilizes statistical standard deviation models rather than recurrent deep neural networks.

7.2 DIFFICULTIES ENCOUNTERED

1. Time-Series Chart Collapses: When telemetry contained only 1 bucket, Visx area series collapsed. Solved via continuous zero-fill aggregation in LogEvent.py.

2. UTC vs. Local Time Discrepancies: Raw UTC timestamps caused the graph axis to display 5.5 hours behind IST (+05:30). Solved by localized formatting in AnimatedAxis tickFormat.

3. Audio Autoplay Restrictions: Browsers block unprompted audio until user gesture. Solved by lazy AudioContext initialization upon first interaction.

7.3 FUTURE ENHANCEMENT SUGGESTIONS

Future enhancements include integrating LSTM/autoencoder deep learning networks for unsupervised UEBA anomaly clustering; graph traversal algorithms for visual attack path reconstruction; and blockchain anchoring for multi-agency forensic evidence custody.

CHAPTER 8

BIBLIOGRAPHY & REFERENCES

8.1 REFERENCE BOOKS

1. Chuvakin, A., Schmidt, K. and Phillips, C. (2013). Logging and Log Management: The Authoritative Guide to Understanding the Concepts to the Implementation. Syngress / Elsevier.

2. Scarfone, K., Jansen, W. and Tracy, M. (2012). Guide to Computer Security Log Management (NIST SP 800-92). National Institute of Standards and Technology.

3. Bejtlich, R. (2013). The Practice of Network Security Monitoring: Understanding Incident Detection and Response. No Starch Press.

4. Sanders, C. and Smith, J. (2014). Applied Network Defense: A Modern Guide to Security Information and Event Monitoring. Syngress.

8.2 OTHER DOCUMENTATION & RESOURCES

1. MITRE Corporation (2024). MITRE ATT&CK® Enterprise Matrix v14. https://attack.mitre.org/

2. SigmaHQ (2024). Generic Signature Format for SIEM Systems (Sigma Rule Specification). https://github.com/SigmaHQ/sigma

3. Krawczyk, H., Bellare, M. and Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication. IETF RFC 2104.

4. Redis Streams Specification: https://redis.io/docs/data-types/streams/

5. Project GitHub Repository: https://github.com/vidhi726/siem

A1 ANNEXURE I: REST API INGESTION CONTRACT

```
POST /v1/logs/ingest
{
 "source_type": "windows_events",
 "source_ip": "192.168.1.105",
 "dest_ip": "10.0.0.1",
 "severity": "high",
 "event_type": "authentication_failure",
 "username": "administrator",
 "hostname": "DC01.CORP.LOCAL",
 "raw": {
  "EventID": 4625,
  "FailureReason": "Bad password"
 }
}
```

A2 ANNEXURE II: SIGMA DETECTION RULE SPECIFICATION

```
title: Multiple Failed Authenticators (SSH / Windows)
id: 5a8e2912-8e10-4c31-9f9b-12d830b201a4
status: production
description: Detects repeated authentication failures indicative of automated credential stuffing.
author: Vidhi Doshi (Adani University)
date: 2026/11/02
logsource:
  category: authentication
  product: enterprise_auth
detection:
  selection:
    event_type: "authentication_failure"
  timeframe: 1m
  condition: selection | count() > 5
level: high
tags:
  - attack.credential_access
  - attack.t1110
```